

Промежуточный отчёт по программному проекту

1. Основные планы и этапы проекта

1.1 Краткое описание проекта

Веб-система для семантического (диалогового) поиска информации в видеопотоках по запросам на естественном языке и фотоизображениям лиц. Интегрируется с внешней системой видеонаблюдения (VMS) как источником видеоматериалов и аналитических событий: пользователь формирует текстовый запрос или прикладывает фото лица и получает релевантные фрагменты видеозаписей с превью, временными метками и глубокими ссылками на видеосегменты во внешней VMS.

Название проекта: «Система поиска информации в видеопотоках по естественному языку» (Natural Language Video Search System).

Цель проекта: автоматический поиск людей, объектов и событий в видеозаписях по текстовому описанию или фотоизображению лица без необходимости ручного просмотра видео.

Краткое описание задач:

1. Анализ и исследование архитектур семантического поиска по видеоданным, выбор базового подхода (baseline).
2. Проектирование серверной архитектуры (LangGraph-агент диалогового поиска) и схем хранения данных (метаданные, эмбединги сущностей, фотоизображения чатов).
3. Развёртывание инфраструктуры (PostgreSQL, Qdrant, Redis, MinIO, TEI, AI-сервис) и контейнеризация через Docker Compose.
4. Разработка backend (HTTP API для авторизации, чат-сессий, диалогового поиска, проксирования медиа из VMS) с автодокументацией.
5. Разработка AI-пайплайна обработки запроса: парсинг через LLM, поиск сущностей в векторной БД, разрешение неоднозначностей через interrupt/resume, поиск по фото через face descriptor VMS.
6. Реализация семантического поиска и ранжирования результатов на основе аналитических событий VMS.
7. Разработка веб-интерфейса (Vue 3 SPA) с поддержкой Server-Sent Events для потокового вывода результата.
8. Тестирование компонентов и стабилизация системы, подготовка программной документации.

Параллельно проведена исследовательская работа (research) по выбору модели для будущей самостоятельной индексации видео: ноутбуки [notebooks/00–05](#) с бенчмарком QVHighlights и сравнением CLIP, X-CLIP, CG-DETR, Qwen3-VL-Embedding-2B/8B (различные стратегии оконной индексации, реранкер).

1.2 Планы и этапы выполнения проекта

Стадии разработки	Этапы работ	Содержание работ	Сроки
Анализ и исследование	Исследование и сравнение архитектур семантического поиска по видеоданным, выбор базового подхода	Описанная архитектура системы и выбранный baseline-метод	01.12.2025 – 31.01.2026
Исследование (research)	Бенчмарк retrieval-моделей на датасете QVHighlights (CLIP, X-CLIP, CG-DETR, Qwen3-VL-Embedding)	Сравнительная таблица моделей по метрикам mean_iou , Recall@K , MR@IoU , MRR , mAP	01.02.2026 – 28.02.2026
Проектирование	Проектирование серверной архитектуры, LangGraph-графа диалогового поиска и схем хранения данных	Архитектурная схема и логические схемы БД	01.03.2026 – 05.03.2026
Инфраструктура хранения	Развёртывание хранилищ метаданных, эмбеддингов сущностей и пользовательских фото	Рабочая инфраструктура в Docker Compose	06.03.2026 – 10.03.2026
Backend	Реализация серверного API для авторизации, чат-сессий и диалогового поиска	HTTP API с автодокументацией (FastAPI + LangGraph)	11.03.2026 – 25.03.2026
Интеграция VMS	Клиент VMS API, проксирование снимков с проверкой авторизации, формирование deep-link	Рабочая интеграция с внешней системой видеонаблюдения	26.03.2026 – 31.03.2026
Поиск и ранжирование	Реализация семантического поиска по сущностям и формирование ответа на основе событий VMS	Поиск с временем первого токена ≤ 5 с и полного ответа ≤ 60 с	в составе backend / интеграции VMS
Frontend	Разработка веб-интерфейса с поддержкой SSE-стриминга и интерактивного выбора кандидатов	Пользовательский интерфейс системы	01.04.2026 – 10.04.2026
Тестирование	Тестирование компонентов и стабилизация системы	Набор unit- и integration-тестов, стабильная версия проекта	11.04.2026 – 28.04.2026

2. Используемый технологический стек и его обоснование

2.1 Перечень используемых технологий

Технология / Инструмент	Описание	Причины выбора
Python 3.11 + FastAPI	Backend HTTP API (авторизация, чат-сессии, проксирование медиа)	Высокая производительность, встроенная автодокументация OpenAPI, асинхронность
LangGraph + LangChain Core	Стейтфул-агент диалогового поиска с поддержкой interrupt/resume	Интерактивное уточнение неоднозначных сущностей; контрольные точки графа в PostgreSQL
PostgreSQL 16 + asyncpg + SQLAlchemy	Хранение пользователей, чатов, сообщений, контрольных точек LangGraph	Надёжная реляционная СУБД, асинхронный драйвер
MinIO (S3)	Объектное хранилище фотоизображений, прикреплённых к сообщениям чата	S3-совместимость, on-prem развёртывание
Qdrant	Векторная БД для эмбедингов сущностей (фамилии, адреса, номерные знаки)	Оптимизирована под семантический поиск с фильтрами, gRPC/HTTP API
Redis	Кеш и список отозванных JWT-токенов	Быстрый in-memory стор, поддержка TTL
Hugging Face TEI (intfloat/multilingual-e5-base)	Сервис генерации текстовых эмбедингов	Готовый сервер с GPU- поддержкой, поддержка русского и английского языков
vLLM (Qwen/Qwen3-8B)	LLM-инференс для парсинга запросов и формирования ответов	OpenAI-совместимый API, structured output, поддержка русского языка
OpenRouter	Альтернативный провайдер LLM	Failover на коммерческие модели в OpenAI- совместимом формате
VMS API (внешняя система видеонаблюдения)	Источник видеоаналитических событий, дескрипторов лиц и снимков	Использование структурированных метаданных VMS вместо самостоятельной индексации сырого видео
Vue 3 + TypeScript + Vite + Pinia + Vue Router + Axios + DOMPurify +	Веб-интерфейс поиска / чата / просмотра	Современный SPA-стек, реактивная работа с SSE,

Технология / Инструмент	Описание	Причины выбора
markdown-it	результатов	безопасный рендеринг ответов LLM
Docker + Docker Compose	Контейнеризация всех сервисов	Воспроизводимое развёртывание на Linux/amd64 и с GPU
SOPS + age	Шифрование переменных окружения в репозитории	Безопасное хранение секретов под git
ClearML	Трекинг экспериментов в исследовательской части	Удобное логирование метрик при бенчмарке retrieval-моделей
PyTorch + Transformers + FAISS (research)	Reference-реализации моделей и векторный индекс в исследовательских ноутбуках	Стандарт ML-экосистемы
Тестирование: pytest (backend / ai), vitest + vue-tsc (frontend)	Автотесты и статическая проверка типов	Контроль качества и регрессий

2.2 Обоснование выбранного технологического стека

Выбранный технологический стек позволяет эффективно реализовать программный продукт с разделением ответственности между компонентами и поддержкой как локального on-prem развёртывания (для интеграции с VMS), так и гибридного со внешними LLM-провайдерами. Использование LangGraph даёт возможность строить агентский пайплайн с интерактивным уточнением неоднозначных сущностей, что критично для предметной области (фамилии, адреса, номерные знаки часто допускают многозначную интерпретацию). Параллельная исследовательская работа на бенчмарке QVHighlights закладывает основу для будущего расширения системы в направлении самостоятельной индексации видеоархивов.

3. Критерии оценивания проекта

Критерий	Описание
Использование инструмента автоматической документации API	Используется (FastAPI автодокументация — /docs Swagger UI, /redoc ReDoc)
Функциональность — Процент выполнения функциональных требований	Реализованы: авторизация / регистрация / удаление аккаунта (JWT + bcrypt + Redis blacklist), чат-сессии (создание / просмотр / удаление), диалоговый поиск с потоковым ответом (SSE), поиск по фото (face descriptor VMS), интерактивное уточнение сущностей (interrupt / resume), интеграция с VMS (deep-link, проксирование снимков), 4 экрана веб-интерфейса. Вне scope

Критерий	Описание
	текущей итерации: личное видеохранилище и встроенный плеер (выводятся через VMS deep-link).
Тестирование — Процент успешных тестов (%)	pytest для backend и AI-сервиса, integration-тесты через make test , vitest для frontend
Соблюдение сроков и плана — Процент выполнения работы в срок (%)	Плановое завершение: не позднее 01.05.2026
Соблюдение сроков и плана — Количество дней отклонения от плана	По текущим этапам — без значимых отклонений
Использование технологического стека — Процент использования функциональности стека (%)	Используются: реляционная БД (PostgreSQL), векторная БД (Qdrant), объектное хранилище (MinIO), кеш (Redis), Docker Compose, веб-интерфейс (Vue 3), автодокументация API (FastAPI), тесты (pytest / vitest), LLM-инференс (vLLM / Qwen3-8B), сервис эмбедингов (TEI / multilingual-e5-base), стейтфул-агент (LangGraph), интеграция с внешним VMS API

4. Особые пометки

Проект является публичной версией коммерческого аутсорс-проекта; интеграция с конкретной VMS закрыта под NDA, поэтому в открытой версии репозитория VMS-клиент работает в режиме mock и реальные креды не приложены. Поисковая часть в проде работает над аналитическими событиями VMS; самостоятельная индексация полнообъёмного видео по эмбедингам остаётся в виде исследовательской работы (**notebooks/**) — направление развития системы.